

TECHCATALYST DE 2026

Terminal Fluency

Build the habits that make every Python, Git, and data workflow more predictable.

The Hartford · TechCatalyst Data Engineering 2026

Week 3 Terminal

Navigate

Inspect

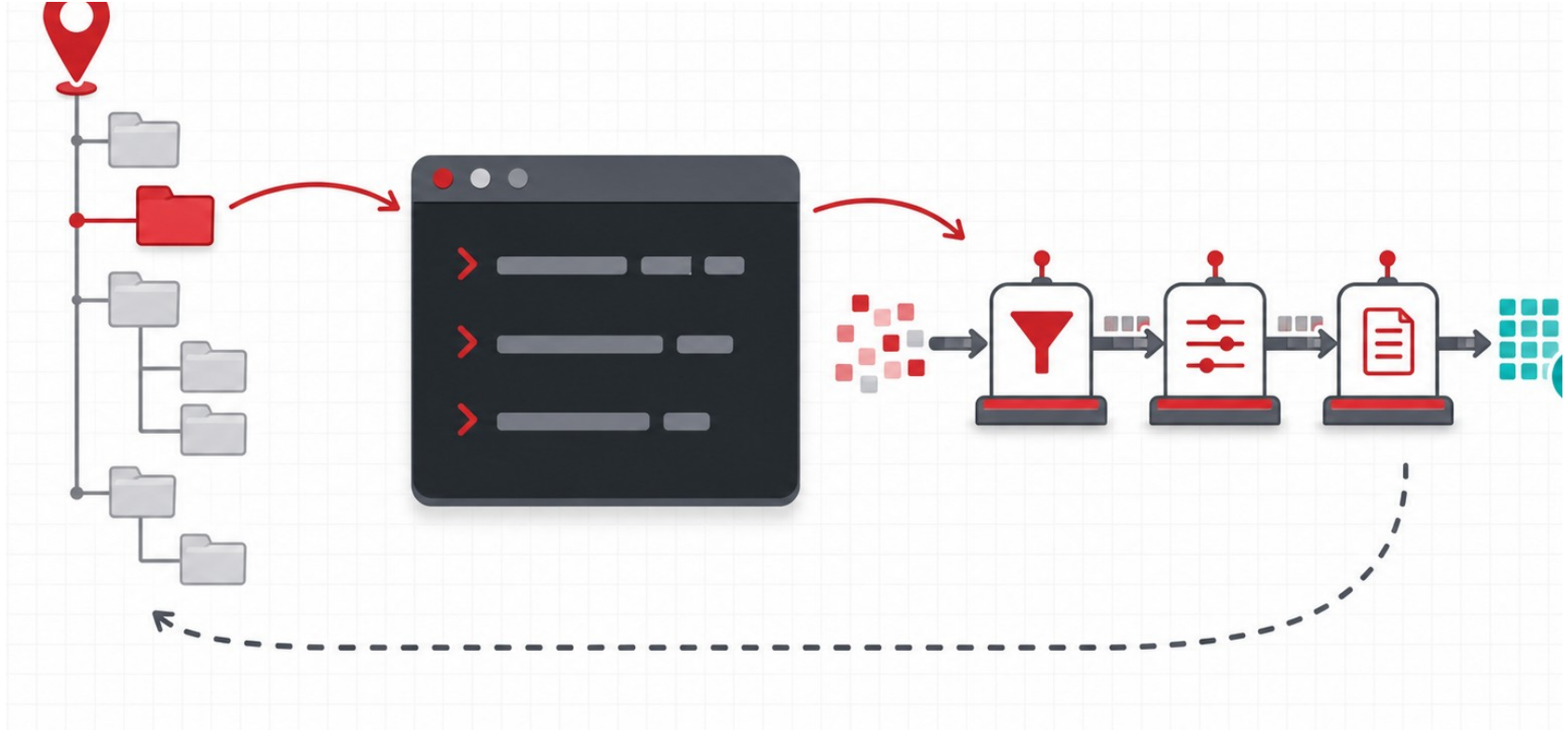
Run

Pipe

Automate

Recover

The shell connects location, commands, and data flow



What you will be able to do

Learning goals	By the end, you can...
Orient	Use pwd and ls before running setup or copy commands
Navigate	Move through absolute and relative paths intentionally
Protect work	Keep provided files separate from student-work
Run safely	Use uv run from the correct daily project
Inspect	Read files, hidden items, permissions, and exit evidence
Compose	Connect small commands with pipes and redirection
Recover	Diagnose the wrong folder or wrong environment quickly
Automate	Turn a repeated sequence into a small shell workflow

Weeks 1 to 4 are an AI-free zone

You cannot review code you could not have written.

What is allowed

- Write and debug your own code
- Use official docs and course references
- Ask peers to explain, not complete
- Call APIs and SDKs when the lesson requires them

Why we do this

- Fluency comes from typing and testing
- Error messages build debugging judgment
- Interviews still test fundamentals
- Later AI use should amplify skills you already own

One workflow, five tools

Terminal

Where you navigate, run, inspect, and automate

Git ♦ GitHub

Local history plus shared collaboration

Python ♦ uv

Logic inside a reproducible daily project

Pandas

The familiar in-memory DataFrame workbench

Polars

An expression-first DataFrame engine with eager and lazy modes

Tools change. Habits stay: inspect, change, verify, and record evidence.

01

Terminal fluency

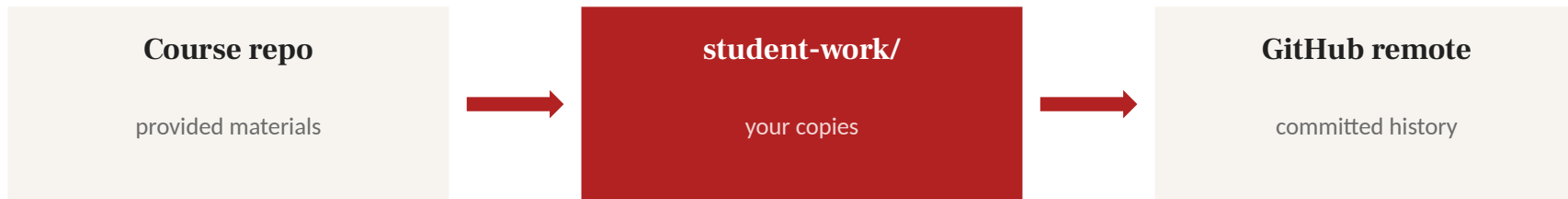
Know where you are, then make one visible change.

Navigate with intention

Command	What it answers
pwd	Where am I right now?
ls -la	What files, folders, and hidden items are here?
cd <path>	Which project or folder am I entering?
mkdir -p <path>	Which workspace am I creating?
cp <source> <target>	What am I copying, and where will I edit it?

Say the path aloud before you run setup or copy commands.

Keep provided files and student work separate



- Pull updates into the provided course paths
- Copy starter files into student-work/week3/day3/
- Edit only your copy, then commit your evidence
- This boundary prevents most pull conflicts

Protect your work from pull conflicts

The rule: run setup commands only after you enter student-work/week3/day3.

- cd into the day folder first
- uv init creates pyproject.toml and .venv there
- uv run automatically uses that project environment
- Select that .venv in VS Code and as the notebook kernel
- If VIRTUAL_ENV points elsewhere, run deactivate and retry

Four habits prevent most workflow chaos

Moment	Check	Question
Before coding	<code>pwd</code>	Am I in the day project?
Before running	<code>uv run ...</code>	Am I using the right environment?
Before committing	<code>git status + git diff</code>	Am I saving only intended work?

Pause when the answer is unclear. A ten-second check is cheaper than a long recovery.

The shell is a feedback loop

Locate: use `pwd` and `ls` to establish the current context before changing anything.

Act: run one focused command, pipe, copy, or environment operation.

Verify: inspect the output, destination, and exit evidence before continuing.

Terminal navigation challenge

Goal: recover from a deliberately confusing folder setup without guessing.

20 min · pairs

- Predict each pwd before running it
- Locate a hidden file and inspect its permissions
- Copy a starter file into student-work/week3/day3
- Run one command through uv and capture the evidence

Stretch: pipe a file listing through sort and save the result without overwriting the source.

Deliverable: a command transcript plus a two-sentence explanation of how you verified the location and environment.

Terminal official references

Reference	Use it for
GNU Coreutils manual	pwd, ls, cp, mkdir, sort, and file utilities
GNU Bash manual	Shell syntax, pipes, redirection, and exit status
uv documentation	Daily projects, environments, and uv run
VS Code terminal docs	Integrated terminal and shell selection
Git documentation	Repository-aware terminal workflow

Checked July 10, 2026. Prefer official documentation when older examples or remembered syntax disagree.

The terminal is your control surface

1

Start with `pwd`. Location is part of every command.

2

Keep learner work under `student-work`, away from provided course paths.

3

Use `uv run` from the daily project so the correct `.venv` is automatic.

4

Compose small commands, then verify output before automating them.

5

When a command fails, inspect the path, environment, input, and exit evidence.

Exit ticket: Which terminal check will you make automatic next week?